Universidad del Valle de Guatemala

Facultad de Ingeniería

Programación Orientada a Objetos

Catedrático: Erick Marroquín

ArenaGT

Análisis y Diseño

Jared Eduardo Guerra Aguirre - 26502

Daniel Alfredo Nájera de Paz - 26961

Ciencias de la Computación y Tecnologías de la Información

Sección 11

11 de Noviembre de 2026

Enlace Repositorio Github: https://github.com/panitodeagua/Lab4Polimorfismo

1. Requisitos funcionales

1. Registrar equipos con código, nombre, empresa y puntos de tabla inicialmente iguales a cero. Rechazar código vacío o repetido sin incorporar el equipo.

2. Conservar al menos dos empresas mediante el atributo empresa de Equipo, no se requiere un catálogo independiente de empresas.

3. Crear encuentros de fútbol sala y baloncesto entre dos equipos ya registrados. Rechazar participantes nulos o con el mismo código.

4. Crear exhibiciones con título, sin equipos, marcador ni puntos de tabla.

5. Crear todo encuentro en SIN_PROGRAMAR y toda exhibición en SIN_PROGRAMAR, sin fecha, hora ni sede asignadas.

6. Programar un evento con fecha válida, hora entera de 0 a 23 y sede no vacía. La operación cambia el estado a PROGRAMADO o PROGRAMADA.

7. Reprogramar encuentros mientras no estén finalizados y exhibiciones programadas, cambiar fecha, hora y sede solo después de todas las validaciones.

8. Rechazar programaciones si otro evento ocupa la misma sede en la misma fecha y hora, sin distinguir encuentros de exhibiciones.

9. Rechazar programaciones si cualquiera de los participantes ocupa otro encuentro en la misma fecha y hora, aunque cambie la sede.

10. Excluir el propio evento del examen de conflictos al reprogramarlo, conservar la programación anterior si la propuesta se rechaza.

11. Aceptar resultados solo de encuentros programados. Rechazar resultados en encuentros sin programación, finalizados o en exhibiciones.

12. Exigir dos enteros no negativos como marcador. Rechazar entradas no numéricas y valores fuera del rango representable por int.

13. En fútbol sala permitir empate, asignar 3 puntos al ganador, 0 al perdedor o 1 a cada equipo en caso de empate.

14. En baloncesto rechazar empate, asignar 2 puntos al ganador y 1 al perdedor.

15. Al aceptar un resultado, guardar ambos marcadores, sumar una sola vez los puntos a los equipos y dejar el encuentro FINALIZADO. No admitir otro resultado ni reprogramación.

16. Mantener una única tabla para ambos deportes. Ordenar por puntos descendentes y en igualdad, por nombre alfabético ascendente usando Comparable<Equipo>.

17. Mostrar en un calendario común solo eventos con programación, incluidos encuentros finalizados y exhibiciones, ordenar por fecha, hora y sede, todas ascendentes.

18. Permitir recorrer Copa mediante Iterable<Programable>, sin exponer las listas internas ni repetir el criterio cronológico.

19. Definir dos interfaces independientes. Programable para calendario y Puntuable para resultados. Exhibicion implementa solo Programable.

20. Usar Encuentro como clase abstracta y una subclase por deporte, despachar las reglas sin instanceof ni comparaciones de clases concretas.

21. Cargar directamente al iniciar al menos cuatro equipos de dos empresas, cuatro encuentros con ambos deportes, dos resultados como mínimo y una exhibición programada.

22. Incluir en la carga un empate de fútbol sala, resultados que muestren la puntuación de ambos deportes y un empate de puntos que se resuelva alfabéticamente.

23. Mostrar un menú para registrar equipos, crear encuentros y exhibiciones, programar/reprogramar, registrar resultados, mostrar tabla y calendario y salir.

24. Mostrar un mensaje de rechazo para entradas inválidas, mantener el menú disponible y conservar el estado anterior de toda operación rechazada.

25. Permitir añadir otros deportes mediante subclases y otros eventos mediante el contrato Programable, sin cambiar la tabla ni el calendario.

2. Clases y responsabilidades

Clase	Tipo	Hereda de / Implementa	Responsabilidad y justificación
Equipo	Concreta	Comparable<Equipo>	Representa un equipo y guarda sus puntos acumulados, define el orden de la tabla
Programable	Interfaz	—	Contrato del calendario, programación, descripción y ocupación de equipos, sin pedir resultados
Puntuable	Interfaz	—	Contrato para guardar un resultado válido y asignar sus puntos de tabla una sola vez
Encuentro	Abstracta	Programable, Puntuable	Centraliza los datos y comportamientos comunes, y delega las reglas específicas de cada deporte a sus subclases
FutbolSala	Concreta	Encuentro	Valida goles y calcula puntos 3, 1, 0
Baloncesto	Concreta	Encuentro	Rechaza empates y calcula puntos 2, 1
Exhibicion	Concreta	Programable	Representa un título y su programación, no tiene equipos ni marcador
Programacion	Concreta final	Comparable<Programacion>	Tiene valor inmutable, valida fecha, hora y sede, concentra una sola vez el orden cronológico
Copa	Concreta	Iterable<Programable>	Guarda equipos y eventos, valida conflictos y muestra consultas sin exponer sus colecciones
Main	Concreta final	—	Guarda la entrada, carga inicial y consola, recibe las entradas y presenta respuestas

3. Atributos

Equipo

Clase	Atributo	Tipo de dato	Visibilidad	Propósito
Equipo	codigo	String	private final	Identidad única normalizada
Equipo	nombre	String	private final	Dato descriptivo obligatorio e inmutable
Equipo	empresa	String	private final	Dato descriptivo obligatorio e inmutable
Equipo	puntosTabla	int	private	Acumulado inicial 0, cambia solo al aceptar resultados

Programable

Clase	Atributo	Tipo de dato	Visibilidad	Propósito
Programable	—	—	—	Definir contratos

Puntuable

Clase	Atributo	Tipo de dato	Visibilidad	Propósito
Puntuable	—	—	—	Definir contratos

Encuentro

Clase	Atributo	Tipo de dato	Visibilidad	Propósito
Encuentro	equipoA	Equipo	private final	Referencia al equipo registrado, participantes diferentes
Encuentro	equipoB	Equipo	private final	Referencia al equipo registrado, participantes diferentes
Encuentro	programacion	Programacion	private	null al crear, referencia inmutable reemplazada al programar
Encuentro	marcadorA	Integer	private	null sin resultado, entero no negativo al finalizar
Encuentro	marcadorB	Integer	private	null sin resultado, entero no negativo al finalizar

FutbolSala

Clase	Atributo	Tipo de dato	Visibilidad	Propósito
FutbolSala	—	—	—	Sin atributos propios, reutiliza el estado definido en Encuentro

Baloncesto

Clase	Atributo	Tipo de dato	Visibilidad	Propósito
Baloncesto	—	—	—	Sin atributos propios reutiliza el estado definido en Encuentro

Exhibicion

Clase	Atributo	Tipo de dato	Visibilidad	Propósito
Exhibicion	titulo	String	private final	Título no vacío de la actividad
Exhibicion	programacion	Programacion	private	null al crear, programación vigente después

Programacion

Clase	Atributo	Tipo de dato	Visibilidad	Propósito
Programacion	fecha	LocalDate	private final	Fecha válida no nula
Programacion	hora	int	private final	Hora entera entre 0 y 23
Programacion	sede	String	private final	Nombre no vacío, sin espacios externos

Copa

Clase	Atributo	Tipo de dato	Visibilidad	Propósito
Copa	equipos	ArrayList<Equipo>	private final	Referencias únicas por código, vacía al inicio
Copa	eventos	ArrayList<Programable>	private final	Lista de todos los eventos, posición + 1 identifica el evento
Copa	puntuables	HashMap<Integer, Puntuable>	private final	Índice de capacidad. ID de evento corresponde al mismo objeto puntuable, no copia datos

Main

Clase	Atributo	Tipo de dato	Visibilidad	Propósito
Main	scanner	Scanner	private static	Lector inicializado en main
Main	copa	Copa	private static	Instancia única coordinada por la consola

4. Métodos

Equipo

Clase	Método	Retorno	Visibilidad	Parámetros	Propósito
Equipo	Equipo	—	public	String codigo, String nombre, String empresa	Constructor, rechaza null o texto vacío, aplica trim, puntosTabla = 0. La unicidad la valida Copa
Equipo	getCodigo	String	public	—	Devuelve codigo
Equipo	getNombre	String	public	—	Devuelve nombre
Equipo	getEmpresa	String	public	—	Devuelve empresa
Equipo	getPuntosTabla	int	public	—	Devuelve puntosTabla
Equipo	sumarPuntos	void	paquete	int puntos	Rechaza negativos antes de sumar y hace la suma de los valores
Equipo	compareTo	int	public	Equipo otro	Implementa. Compara puntos descendentes y luego nombre ascendente con String.compare To, no resta enteros

Programable

Clase	Método	Retorno	Visibilidad	Parámetros	Propósito
Programable	getProgramacion	Programacion	public abstract	—	Abstracta, devuelve null si todavía no se programó.
Programable	programar	void	public abstract	Programacion propuesta, Copa copa	Abstracta, valida estado y conflictos en Copa antes de cambiar la programación
Programable	getCodigosEquipos	Set<String>	public abstract	—	Abstracta, conjunto inmutable de códigos ocupados, vacío si no hay equipos
Programable	getDescripcion	String	public abstract	—	Abstracta, descripción legible para el calendario
Programable	getEstado	String	public abstract	—	Abstracta, nombre del estado

Puntuable

Clase	Método	Retorno	Visibilidad	Parámetros	Propósito
Puntuable	registrar Resultado	void	public abstract	int a, int b	Abstracta, valida el resultado, reparte puntos y finaliza de forma indivisible, rechaza sin cambios

Encuentro

Clase	Método	Retorno	Visibilidad	Parámetros	Propósito
Encuentro	Encuentro	—	protected	Equipo equipo A, Equipo equipoB	Constructor, rechaza null e igualdad de códigos, asigna referencias y deja programación y marcadores en null
Encuentro	getEquipoA	Equipo	public final	—	Devuelve el primer participante

Clase	Método	Retorno	Visibilidad	Parámetros	Propósito
Encuentro	getEquipoB	Equipo	public final	—	Devuelve el segundo participante
Encuentro	getMarcadorA	Integer	public final	—	Devuelve marcador A o null
Encuentro	getMarcadorB	Integer	public final	—	Devuelve marcador B o null
Encuentro	getProgramacion	Programacion	public final	—	Implementa, devuelve la programación actual o null
Encuentro	programar	void	public final	Programacion propuesta, Copa copa	Implementa, rechaza finalizado o null, pide validarProgramacion a Copa, asigna solo si todo es válido
Encuentro	getCodigos Equipos	Set<String>	public final	—	Implementa, devuelve Set.of con los dos códigos.
Encuentro	getEstado	String	public final	—	Implementa, FINALIZADO si hay marcadores, PROGRAMADO si hay programación, SIN_PROGRAMAR en otro caso
Encuentro	registrarResultado	void	public final	int a, int b	Implementa, pide PROGRAMADO, valida marcador, calcula ambos incrementos y verifica sumas, después actualiza puntos y marcadores.
Encuentro	getDescripcion	String	public abstract	—	Abstracta, cada deporte identifica su nombre, equipos y marcador si existe
Encuentro	validarMarcador	void	protected	int a, int b	Base, valida que ambos marcadores sean enteros no negativos. Baloncesto sobrescribe el método y usa super.validarMarcador para añadir la restricción de no permitir empates

Clase	Método	Retorno	Visibilidad	Parámetros	Propósito
Encuentro	calcularPuntos	int[]	protected abstract	int a, int b	Abstracta, devuelve dos incrementos no negativos, [A, B], sin cambiar equipos

FutbolSala

Clase	Método	Retorno	Visibilidad	Parámetros	Propósito
FutbolSala	FutbolSala	—	public	Equipo equipoA, Equipo equipoB	Constructor, llama super (equipoA, equipoB), no añade atributos adicionales
FutbolSala	calcularPuntos	int[]	protected	int a, int b	Sobrescribe, devuelve [3,0], [1,1] o [0,3]
FutbolSala	getDescripcion	String	public	—	Sobrescribe, identifica el deporte y los participantes, añade marcador solo si está finalizado

Baloncesto

Clase	Método	Retorno	Visibilidad	Parámetros	Propósito
Baloncesto	Baloncesto	—	public	Equipo equipo A, Equipo equipoB	Constructor, llama super(equipoA, equipoB), no añade atributos adicionales
Baloncesto	validarMarcador	void	protected	int a, int b	Sobrescribe, llama super.validarMarcador(a, b) y rechaza a == b
Baloncesto	calcularPuntos	int[]	protected	int a, int b	Sobrescribe, devuelve [2,1] o [1,2] para marcador validado
Baloncesto	getDescripcion	String	public	—	Sobrescribe, identifica el deporte y los participantes, añade marcador solo si está finalizado

Exhibicion

Clase	Método	Retorno	Visibilidad	Parámetros	Propósito
Exhibicion	Exhibicion	—	public	String titulo	Constructor, rechaza null o vacío, asigna trim del título y programación null
Exhibicion	getTitulo	String	public	—	Devuelve el título
Exhibicion	get Programacion	Programacion	public	—	Implementa, devuelve programación o null
Exhibicion	programar	void	public	Programacion propuesta, Copa copa	Implementa, rechaza null, valida en Copa y reemplaza la programación solo si no hay conflictos
Exhibicion	getCodigos Equipos	Set< String>	public	—	Implementa, devuelve Set.of(), conjunto vacío inmutable.
Exhibicion	getDescripcion	String	public	—	Implementa, devuelve el título de la exhibición
Exhibicion	getEstado	String	public	—	Implementa, devuelve SIN_PROGRAMAR o PROGRAMADA según haya programación

Programacion

Clase	Método	Retorno	Visibilidad	Parámetros	Propósito
Programacion	Programacion	—	public	LocalDate fecha, int hora, String sede	Constructor, rechaza fecha null, hora fuera de 0–23 y sede null o vacía, guarda sede.trim()
Programacion	getFecha	Local Date	public	—	Devuelve el valor inmutable de fecha
Programacion	getHora	int	public	—	Devuelve el valor inmutable de hora

Clase	Método	Retorno	Visibilidad	Parámetros	Propósito
Programacion	getSede	String	public	—	Devuelve el valor inmutable de sede
Programacion	compareTo	int	public	Programacion otra	Implementa, compara fecha ascendente, hora ascendente y sede por String.compareTo, en ese orden

Copa

Clase	Método	Retorno	Visibilidad	Parámetros	Propósito
Copa	Copa	—	public	—	Constructor, inicializa las dos listas y el mapa vacíos
Copa	registrarEquipo	void	public	Equipo equipo	Rechaza null o código existente, agrega al finalizar la validación
Copa	buscarEquipo	Equipo	public	String codigo	Busca código.trim(), lanza Illegal ArgumentException si falta, es vacío o no existe
Copa	agregar Encuentro	int	public	Encuentro encuentro	Comprueba participantes registrados, evento nuevo y sin programación, agrega evento e índice Puntuable, Devuelve ID
Copa	agregarEvento	int	public	Programable evento	Registra actividad solo de calendario, no nula, nueva y sin programación, devuelve ID. Los encuentros usan agregarEncuentro
Copa	buscarEvento	Programable	public	int id	Valida 1 ≤ id ≤ tamaño, devuelve eventos.get(id - 1)
Copa	listarEventos	List<Programable>	public	—	Devuelve List.copyOf(eventos), en orden de registro para seleccionar ID
Copa	programar Evento	void	public	int id, LocalDate fecha, int hora, String sede	Construye Programacion y llama programar(propuesta, this) en el evento. Sirve también para reprogramar

Clase	Método	Retorno	Visibilidad	Parámetros	Propósito
Copa	validar Programacion	void	public	Programable evento, Programacion propuesta	Pide evento registrado por identidad y propuesta no nula. Excluye el mismo evento, comprueba sede y equipos a igual fecha/hora
Copa	registrar Resultado	void	public	int id, int a, int b	Busca Puntuable en el índice, rechaza ID sin esa capacidad y da registrar Resultado(a, b)
Copa	getTabla Posiciones	List<Equipo>	public	—	Pide una copia con Equipo.compareTo, retorna lista no modificable
Copa	iterator	Iterator< Programable>	public	—	Implementa, filtra eventos con programación, ordena copia con Comparator.comparing(Programable::getProgramacion) y devuelve iterador no modificable

Main

Clase	Método	Retorno	Visibilidad	Parámetros	Propósito
Main	Main	—	private	—	Constructor, impide instancias, la consola usa miembros estáticos
Main	main	void	public static	String[] args	Inicializa Scanner y Copa, carga datos, ejecuta menú y cierra el lector al salir
Main	cargarDatosIniciales	void	private static	—	Construye los datos iniciales y los registra, programa y finaliza por los mismos métodos del dominio
Main	ejecutarMenu	void	private static	—	Repite opciones hasta 0, captura errores de dominio y formato, muestra mensajes y sigue el ciclo
Main	mostrarMenu	void	private static	—	Muestra 1 registrar equipo, 2 crear encuentro, 3 crear exhibición, 4 programar/reprogramar, 5 resultado, 6 tabla, 7 calendario, 0 salir

Clase	Método	Retorno	Visibilidad	Parámetros	Propósito
Main	registrarEquipo	void	private static	—	Lee código, nombre y empresa, construye Equipo y llama copa.registrarEquipo
Main	crearEncuentro	void	private static	—	Lee deporte y códigos, obtiene equipos de Copa, crea FutbolSala o Baloncesto y llama agregarEncuentro
Main	crearExhibicion	void	private static	—	Lee título, construye Exhibicion y llama agregarEvento
Main	programarEvento	void	private static	—	Lista IDs y estados, lee ID, fecha, hora y sede y llama Copa.programarEvento
Main	registrarResultado	void	private static	—	Lista IDs, lee ID y dos enteros y llama Copa.registrarResultado
Main	mostrarTabla	void	private static	—	Recorre getTablaPosiciones y muestra código, nombre, empresa y puntosTabla
Main	mostrarCalendario	void	private static	—	Usa for-each sobre Copa, muestra descripción, fecha, hora, sede y estado sin decidir el deporte
Main	leerTexto	String	private static	String mensaje	Muestra mensaje y lee línea, EOF lanza NoSuchElementException para salida controlada
Main	leerEntero	int	private static	String mensaje	Convierte leerTexto con Integer.parseInt, reintenta ante NumberFormatException
Main	leerFecha	LocalDate	private static	String mensaje	Convierte formato ISO aaaa-mm-dd, reintenta ante DateTimeParseException, no acepta fechas inexistentes

Los objetos reciben sus valores iniciales mediante sus constructores. Las clases FutbolSala y Baloncesto usan super(equipoA, equipoB) para inicializar los atributos heredados de Encuentro. La clase abstracta valida los participantes y establece la programación y los marcadores inicialmente en null. Cada subclase implementa las reglas específicas de su deporte.

5. Reglas de negocio, estados y transiciones

Deporte	Victoria	Empate	Derrota	Validación
Fútbol sala	3	1 por equipo	0	Dos enteros ≥ 0, se permite igualdad
Baloncesto	2	No permitido	1	Dos enteros ≥ 0 y diferentes

Estados de los encuentros

Estado inicial	Operación	Condición	Estado final	Efecto
No existe	Crear	Equipos registrados y distintos	SIN_PROGRAMAR	Marcadores y programación null
SIN_PROGRAMAR	Programar	Fecha, hora y sede válidas, sin conflictos de sede ni equipos	PROGRAMADO	Guarda programación completa
PROGRAMADO	Reprogramar	Nueva fecha, hora y sede válidas, sin conflictos de programación	PROGRAMADO	Reemplaza programación, no suma puntos
PROGRAMADO	Registrar resultado	Marcador válido según deporte	FINALIZADO	Guarda marcador y suma ambos incrementos
SIN_PROGRAMAR	Registrar resultado	Siempre inválida	SIN_PROGRAMAR	Rechazo sin cambios
FINALIZADO	Resultado o reprogramación	Siempre inválida	FINALIZADO	Rechazo, guarda resultado y puntos
Cualquier estado	Operación inválida	Datos, estado o conflictos inválidos	El mismo	No cambia ningún dato

Estados de las exhibiciones

Estado inicial	Operación	Condición	Estado final	Efecto
No existe	Crear	Título válido	SIN_PROGRAMAR	Programación null
SIN_PROGRAMAR	Programar	Horario válido y sede libre	PROGRAMADA	Se añade al calendario

Estado inicial	Operación	Condición	Estado final	Efecto
PROGRAMADA	Reprogramar	Horario válido y sede libre	PROGRAMADA	Reemplaza programación
Cualquier estado	Registrar resultado	No tiene contrato Puntuable	El mismo	Copa rechaza ID, no modifica puntos
Cualquier estado	Programación inválida	Fecha, hora, sede o conflicto inválido	El mismo	Guarda la programación anterior

6. Herencia, interfaces y comportamientos polimórficos

Método o contrato	**Clase base o interfaz**	**Implementado por**	**Justificación**
Participantes, marcador y ciclo de resultado	Encuentro	FutbolSala y Baloncesto por herencia	La clase abstracta impide encuentros genéricos
programar, consultas de calendario	Programable	Encuentro y Exhibicion	Comparte una capacidad sin crear herencia artificial entre eventos distintos
registrarResultado	Puntuable	Encuentro	Encapsula validación, reparto y finalización. Exhibicion no tiene de esta capacidad
validarMarcador	Encuentro	Base común, Baloncesto amplía	La clase base rechaza marcadores negativos y Baloncesto agrega la restricción de no permitir empates
calcularPuntos	Encuentro	FutbolSala y Baloncesto	Despacho dinámico decide los incrementos, sin condicionales de tipos
getDescripcion	Programable y Encuentro	Los dos deportes y Exhibicion	La consola muestra cada objeto con el mismo contrato
compareTo	Comparable<Equipo>	Equipo	Permite ordenar los equipos de mayor a menor según sus puntos acumulados y en caso de empate, por nombre en orden alfabético.
compareTo	Comparable<Programacion>	Programacion	Única definición de fecha, hora y sede ascendentes
iterator	Iterable<Programable>	Copa	Filtra y ordena una copia, for-each no conoce el almacenamiento
getCodigosEquipos	Programable	Encuentro y Exhibicion	El conjunto vacío de exhibición evita preguntar por tipos al validar conflictos

Se usan dos interfaces porque no todos los eventos participan en competencias. Si Exhibicion implementara Puntuable, tendría que dar operaciones de resultados y asignación de puntos que no le corresponden.

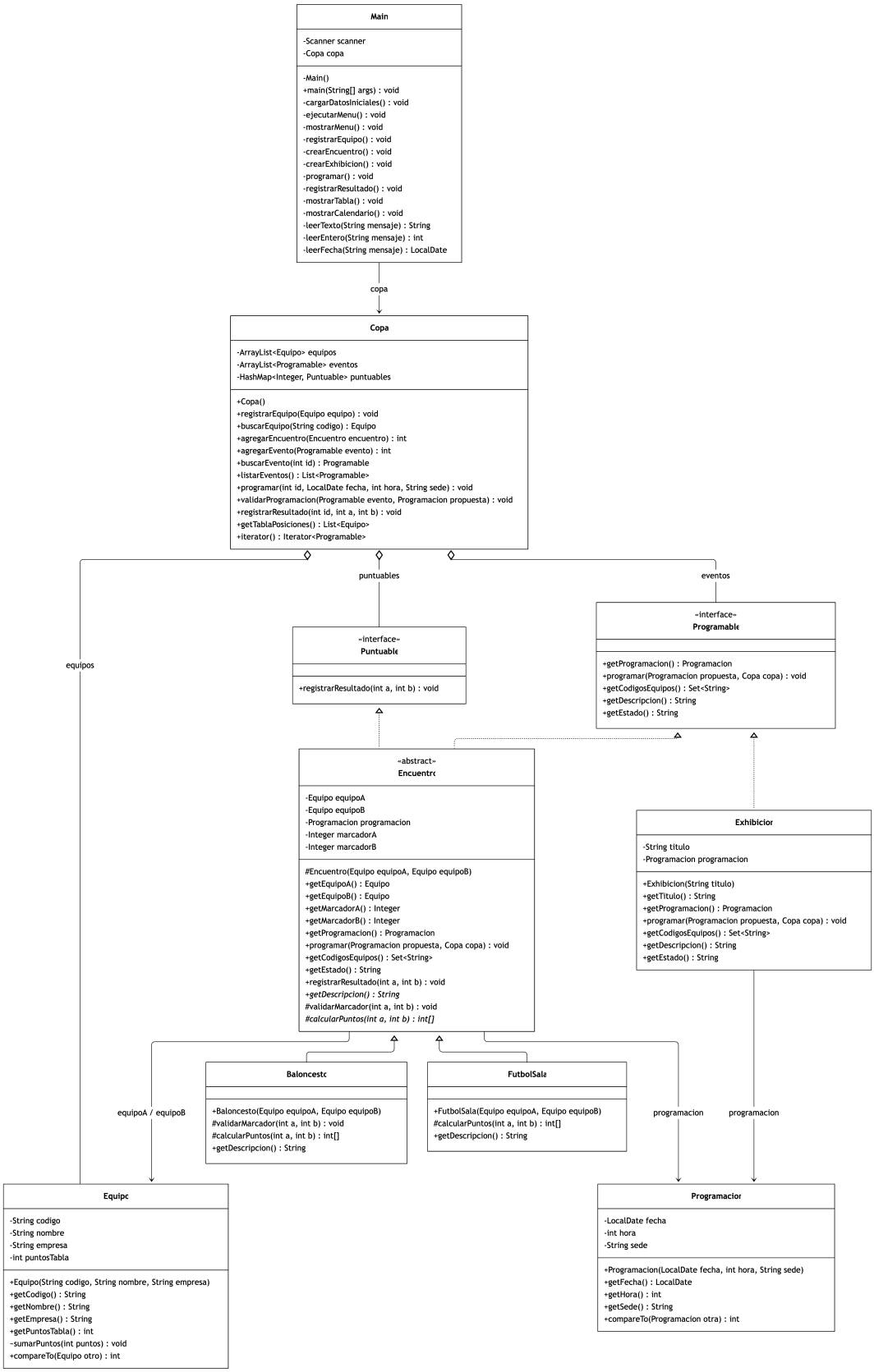
7. Extensibilidad del sistema

Para añadir voleibol se crea Voleibol extends Encuentro, con constructor Voleibol(Equipo equipoA, Equipo equipoB) que invoca super. Implementa calcularPuntos(int a, int b), sobrescribe validarMarcador(int a, int b) invocando primero super, e implementa getDescripcion(). Los marcadores pueden representar sets ganados. Como ejemplo, se admitirían 3–0, 3–1 y 3–2 o sus inversos, se repartirían 3/0 en los dos primeros casos y 2/1 en el último. Esta política tendría sus propias pruebas antes de incorporarse.

Solo se agrega la clase y se amplía la selección de creación y la carga de demostración en Main. agregarEncuentro ya acepta cualquier subclase de Encuentro. No se modifica registrarResultado, getTablaPosiciones, iterator, Equipo, Programacion ni el análisis de conflictos. La incorporación no añade condicionales por deporte en la lógica original. Las reglas de voleibol se ejecutan por los mismos métodos protegidos que utilizan los deportes originales.

Un nuevo evento sin resultados implementa Programable y se registra con agregarEvento, devuelve los códigos que ocupa o un conjunto vacío. El calendario se queda igual.

8. Diagrama UML



9. Evidencia de pruebas

Se realizaron pruebas de las reglas de puntuación, los estados, la tabla de posiciones, el calendario y las operaciones inválidas. También se comprobaron las victorias de ambos participantes en cada deporte y las entradas inválidas de la consola.

En las pruebas de puntuación, cada caso comienza con ambos equipos en cero puntos, salvo que se indique una acumulación previa.

N.º	Caso y datos de entrada	Resultado esperado	Resultado obtenido
1	Fútbol sala: A vence 3–1 a B en un encuentro programado.	A suma 3 puntos y B suma 0. El encuentro finaliza.	A: 3; B: 0. Estado FINALIZADO.
2	Fútbol sala: A pierde 1–3 contra B.	A suma 0 puntos y B suma 3.	A: 0; B: 3.
3	Fútbol sala: empate 2–2.	Cada equipo suma 1 punto.	A: 1; B: 1.
4	Baloncesto: A vence 78–74 a B.	A suma 2 puntos y B suma 1.	A: 2; B: 1.
5	Baloncesto: A pierde 74–78 contra B.	A suma 1 punto y B suma 2.	A: 1; B: 2.
6	Los mismos equipos participan en ambos deportes: A gana en fútbol sala y en baloncesto.	Los puntos se acumulan en una sola tabla: A obtiene 5 y B obtiene 1.	A: 5; B: 1 en la tabla general.
7	Tabla con Cóndores: 2 puntos; Aurora, Búhos y Delfines: 1 punto cada uno.	Orden descendente por puntos y alfabético en el empate.	Cóndores, Aurora, Búhos y Delfines.
8	Calendario con una exhibición a las 08:00 y encuentros a las 09:00 en Cancha y Gimnasio, todos en la misma fecha.	Exhibición primero; luego los encuentros, ordenados por sede al coincidir fecha y hora.	Exhibición, encuentro en Cancha y encuentro en Gimnasio.
9	Comparar 06/11/2026 a las 23:00 con 07/11/2026 a las 00:00.	El evento del 6 de noviembre aparece primero.	Se respetó el orden por fecha.
10	Calendario con encuentros finalizados y otro sin programación.	Incluir los finalizados y excluir el que no tiene programación.	Los finalizados aparecieron; el encuentro sin programar quedó excluido.
11	Reprogramar una exhibición del 06/11/2026 a las 08:00 al 07/11/2026 a las 23:00, en Plaza.	Guardar la nueva programación y conservar el estado PROGRAMADA.	Fecha, hora y sede actualizadas; estado PROGRAMADA.
12	Reprogramar un encuentro conservando	Aceptar: el evento no debe entrar en	Programación aceptada; estado PROGRAMADO.

	su misma fecha, hora y sede.	conflicto consigo mismo.	
13	Registrar un equipo con código vacío y otro con un código ya existente.	Rechazar ambos registros sin incorporar equipos.	Registros rechazados; se conservaron los cuatro equipos originales.
14	Crear un encuentro usando el mismo equipo como A y B.	Rechazar la creación.	Se lanzó el rechazo; no se agregó el encuentro.
15	Registrar marcador −1–0 en un encuentro programado.	Rechazar el marcador y conservar los puntos y el estado anterior.	Marcador rechazado; el encuentro siguió programado y sin resultado.
16	Registrar empate 10–10 en baloncesto.	Rechazarlo sin asignar puntos ni guardar marcadores.	Equipos con 0 puntos; marcador sin registrar.
17	Registrar resultado 0–0 en un encuentro sin programar.	Rechazarlo y conservar el estado SIN_PROGRAMAR.	Operación rechazada; no se asignaron puntos.
18	Registrar un segundo resultado en un encuentro finalizado con empate 2–2.	Rechazarlo y conservar el resultado y los puntos originales.	Se mantuvo el 2–2 y 1 punto para cada equipo.
19	Reprogramar un encuentro finalizado.	Rechazarlo sin cambiar su programación, resultado ni puntos.	Operación rechazada; se conservaron los datos anteriores.
20	Registrar un resultado en una exhibición.	Rechazarlo porque la exhibición no implementa Puntuable.	Resultado rechazado; la tabla permaneció igual.
21	Reprogramar una exhibición a una sede ocupada por un encuentro en la misma fecha y hora.	Rechazar la propuesta y conservar su programación anterior.	Conflicto detectado; la exhibición conservó su horario de las 08:00.
22	Programar otro encuentro a las 09:00 con un equipo que ya juega a esa hora, utilizando otra sede.	Rechazarlo aunque cambie la sede.	Conflicto de equipo detectado; el encuentro permaneció sin programar.
23	Introducir horas −1, 24 o 25.	Rechazar las horas fuera del intervalo de 0 a 23.	Horas rechazadas; no se guardó una programación inválida.
24	Introducir «letras» o 2147483648 donde se solicita un entero.	Mostrar un mensaje y solicitar nuevamente el dato sin cerrar el programa.	Se mostraron los mensajes y continuó la lectura.
25	Introducir la fecha inexistente 2026-02-30.	Rechazarla y solicitar otra fecha.	La consola mostró el rechazo y permitió corregirla.

26	Consultar los IDs de evento 0 y 999.	Rechazarlos por no corresponder a eventos registrados.	Ambos IDs fueron rechazados.
27	Registrar un resultado que incrementaría los puntos de un equipo por encima de Integer.MAX_VALUE.	Rechazar toda la operación sin actualizar parcialmente al otro equipo.	Se conservaron los puntos de ambos equipos, los marcadores vacíos y el estado PROGRAMADO.
28	Intentar vaciar las listas devueltas o eliminar un evento mediante el iterador del calendario.	Impedir la modificación de las colecciones internas.	Las operaciones lanzaron UnsupportedOperationException.

Conservación de la tabla ante operaciones rechazadas**.** En las pruebas de conflictos de programación no se ejecutó el reparto de puntos. El empate inválido de baloncesto conservó los puntos iniciales; el resultado repetido conservó los puntos ya asignados; y el desbordamiento se rechazó antes de modificar cualquiera de los dos equipos.